Exercise 1: autocovariance and power/coherence spectra of stochastic processes

a) Random walk: $X_t = \alpha X_{t-1} + \epsilon_t$ with mean 0 , $\epsilon_t \sim N(0, \sigma^2)$, $\alpha = 1$

For a random walk, the variance grows with time. Thus: $\text{Var}(X_t) = t\sigma^2$

And: $\text{Var}(X_t) \neq \text{Var}(X_{t+\tau}) \longrightarrow R(s,t) = \min\{s,t\}\sigma_\epsilon^2$

To compute the autocovariance, it cannot be as a function of τ but of a chosen t_1, t_2 $\Rightarrow$ due to the variance increasing & how $\{\epsilon_t\}$ are uncorrelated r.v's.

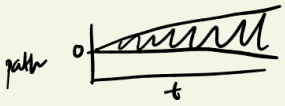
In the same vein, for a random walk with drift, the function is:

$X_t = \mu t + \alpha X_{t-1} + \epsilon_t$ where $\mu \neq 0$, $\alpha = 1$, $\epsilon_t \sim N(0, \sigma^2)$.

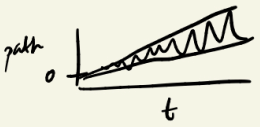
And likewise, the variance is $t\sigma^2$. thus $\text{Var}(X_t) \neq \text{Var}(X_{t+\tau})$

Furthermore, the mean is $\mathbb{E}(X_t) = \mu t$. We cannot define the autocovariance in this case, for the same reason as for a random walk.

Shape for random walk:



Shape for random walk w/ drift:



b) The deterministic component is the harmonic process (f_1, f_2 & f_3).

This is because the harmonic process can represented as a fourier series.

Thus we have: $X_t = \sum_{n=1}^{3} a_n \sin(2\pi f_n t + \phi_n)$. where the a_n is the amplitude & the ϕ_n is the phase.

Once we know the amplitudes & phases, everything should be predictable for future times.

However, the AR(1) process is not deterministic, as the process's equation is:

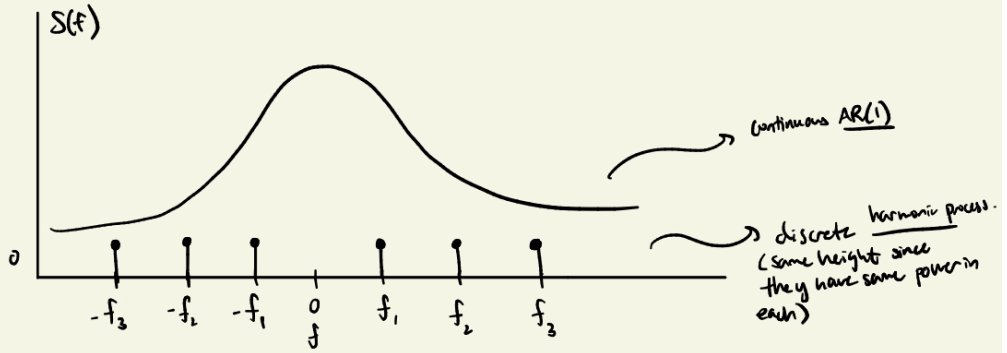
$$X_t = a X_{t-1} + \epsilon_t \quad , a > 0.$$

ϵ_t is white noise, which is inherently unpredictable, even knowing X_{t-1} and everything before, due to the randomness from the ϵ_t.

The harmonic process contributes to the discrete part of the spectrum: it has a "line" spectrum the sinusoidal frequencies (f_1, f_2, f_3) produce dirac delta spikes. $\Rightarrow$ real signal has a $\oplus$ & $\ominus$ frequency.

because $x_0(t) = 2\cos(\omega_0 t) = \left[e^{i\omega_0 t} + e^{i(-\omega_0)t}\right]$.

The AR(1) component contributes to the continuous part of the spectrum: it has a corresponding power spectral density - $S(f)$



c) $f_s = 2 \frac{\text{kilosamples}}{s} = 2\,\text{kHz} = 2000\ \text{samples/s}$

Duration $\Rightarrow 0.2\text{s} = T$

$N = f_s T = 2000 \cdot 0.2 = 400$ samples.

The frequency resolution is basically: $\Delta f = \frac{1}{T} \Rightarrow$ spacing between fourier frequencies.

$\Delta f = \frac{1}{T} = \frac{1}{0.2\text{s}} = 5\,\text{Hz}$

Zero padding does not increase the true resolution beyond $\frac{1}{T}$ or $\frac{f_s}{N}$. Although we create more datapoints (w/ adding more zeros), it merely interpolates the frequency spectrum which creates a smoother visual representation of the discrete fourier. (appears to increase though).

The maximum frequency of the spectral estimate is given by the Nyquist frequency:

$$f_{max} = \frac{f_s}{2} = \frac{2000}{2} = \boxed{1000\ \text{Hz}}$$

d) We know that coherence is defined as the normalized cross-spectrum:

$$C_{xx}(f) = \frac{|S_{xy}(f)|^2}{S_{xx}(f)\,S_{yy}(f)} = 0.3$$

A single-taper estimate has 2 DOF $\Rightarrow$ w/ one Hann taper

$\rightarrow$ High variance, strong bias & extremely unreliable.

This is an extremely noisy estimator of cross-spectrum.

With only 2 DOF, we would trend towards 1. regardless of the true coherence (0.3).

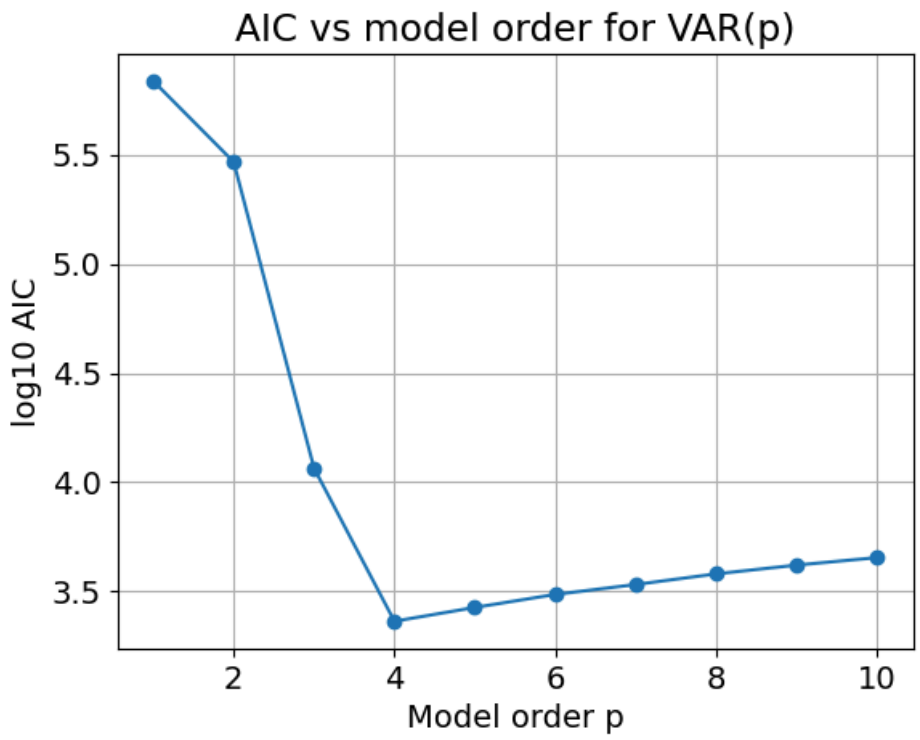
Based on only 1 taper, the estimation seems like it would be dominated by noise.

Exercise 2: mystery circuit -- spectral power, coherence and Granger causality

(a)

```
# specifying the path to the .mat file
mat_file_path = 'mystery_circuit_data_py.mat'

# load .mat file
data =
sio.loadmat(r'c:\Users\Lyons\Desktop\NEUR2110\Final_Exam\mystery_circuit_data_py.mat')

# getting a better idea of the shape of arrays
print(f"variables in .mat: {data.keys()}")

# define variables
N = data['N']
X = data['X']
fs = data['Fs'][0,0]
nTrials = data['nTrials'][0,0]
nChns = data['nChns'][0,0]

print(f"N shape: {N.shape}, X shape: {X.shape}, fs: {fs}, nTrials: {nTrials}, nChns:
{nChns}")
```

```
max_p = 10
AIC_vals = np.zeros(max_p)
M = N * nTrials  # total number of samples per channel


for p in range(1, max_p+1):
    # fit VAR(p) by maximum entropy
    A, SIGMA, E = var_maxent(X, p)

    # determinant of residual covariance
    DSIG = np.linalg.det(SIGMA)

    # maximized log-likelihood (per formula)
    L = -(M/2.0) * np.log(DSIG)

    # AIC for this p (their formula)
    AIC = -2.0*L + 2.0*p*(nChns**2) * float(M/(M - p - 1.0))
    AIC_vals[p-1] = AIC


# plot AIC in log10 scale vs model order
orders = np.arange(1, max_p+1)

plt.figure()
plt.plot(orders, np.log10(AIC_vals), marker='o')
plt.xlabel('Model order p')
plt.ylabel('log10 AIC')
plt.title('AIC vs model order for VAR(p)')
plt.grid(True)
plt.show()

# select optimal order
p_opt = orders[np.argmin(AIC_vals)]
print("Optimal model order (AIC):", p_opt)
```



Optimal model order (AIC): 4

(b) The VAR(4) model is stationary because the eigenvalues of the companion matrix constructed from the estimated VAR coefficients all lie strictly inside the unit circle. Specifically, the maximum absolute eigenvalue of the companion matrix is 0.954, which is less than 1. Therefore, the VAR(4) process satisfies the stability condition and is stationary.

```
A_opt, SIGMA_opt, E_opt = var_maxent(X, p_opt)
p_opt, nChns, _ = A_opt.shape
C_top = np.hstack([A_opt[k] for k in range(p_opt)])

if p_opt > 1:
    C_bottom = np.eye(nChns*(p_opt-1), nChns*p_opt)    # shape ((p-1)*n, p*n)
    C = np.vstack([C_top, C_bottom])                   # full companion matrix
else:
    C = C_top
print("C shape:", C.shape)
eigvals = np.linalg.eigvals(C)
max_eig = np.max(np.abs(eigvals))
print("\nMax |eigenvalue| =", max_eig)

if max_eig < 1:
    print(f"\nThe VAR({p_opt}) model is stationary.")
else:
    print(f"\nThe VAR({p_opt}) model is not stationary.")
```

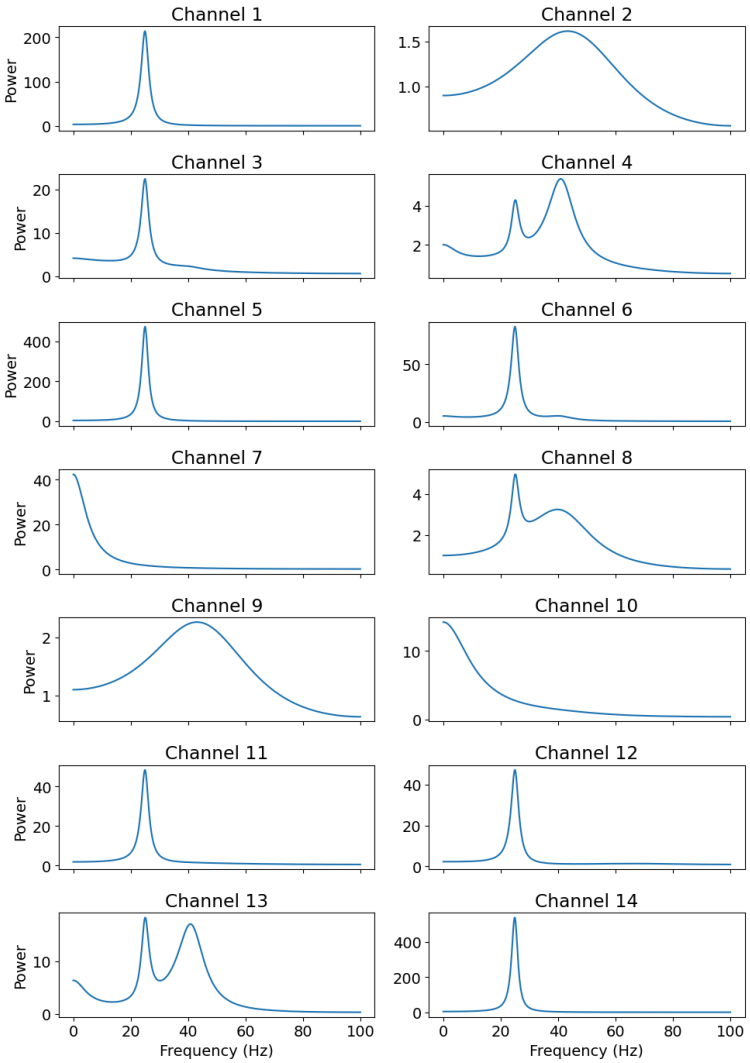
C shape: (56, 56)

Max |eigenvalue| = 0.9540617483670679

The VAR(4) model is stationary.

(c) Overall, the VAR-based parametric spectra show a prominent narrowband oscillation around ~30 Hz across many channels (especially 1, 3, 5, 6, 11, 12, 14), while channels 2 and 9 show broader band-limited humps in the ~30–40 Hz range. Channels 4 and 13 exhibit clear two-peak structure (~25–30 Hz and ~40–45 Hz), and channels 7 and 10 are dominated by low-frequency power with a steep decay, consistent with slower fluctuations rather than oscillatory activity.

```
f_min, f_max = 0.0, 100.0
nFreq = 301
freqs = np.linspace(f_min, f_max, nFreq)

S_f = np.zeros((nFreq, nChns, nChns), dtype=complex)
I = np.eye(nChns, dtype=complex)

for i, f in enumerate(freqs):
    # A(f) = I - sum_{k=1}^p A_k * exp(-j 2π f k / fs)
    Af = I.copy()
    for k in range(p_opt):
        Af -= A_opt[k] * np.exp(-1j * 2.0 * np.pi * f * (k + 1) / fs)
    # Transfer function H(f) = A(f)^(-1)
    Hf = np.linalg.inv(Af)
    # Spectral density: S(f) = H(f) Σ H(f)^H
    S_f[i] = Hf @ SIGMA_opt @ Hf.conj().T

fig, axes = plt.subplots(7, 2, figsize=(10, 14), sharex=True)
axes = axes.ravel()

for ch in range(nChns):  # channels 0 - 14
    ax = axes[ch]
    Pxx = np.real(S_f[:, ch, ch])  # auto-spectrum of channel ch
    ax.plot(freqs, Pxx)
    ax.set_title(f'Channel {ch+1}')
    if ch >= nChns - 2:
        ax.set_xlabel('Frequency (Hz)')
    if ch % 2 == 0:
        ax.set_ylabel('Power')

plt.tight_layout()
plt.show()
```



(d) Yes, multitaper and parametric VAR-based spectral coherences largely agree in both the presence and locations of coherence peaks across channel pairs, indicating that the VAR model is capturing the dominant cross-channel dependencies present in the data.

Across both methods, the spectral coherences show pronounced low-frequency peaks, primarily concentrated in the theta/alpha range (roughly 5–15 Hz), with some channel pairs also exhibiting broader low-frequency structure below ~10 Hz. These peaks appear consistently in the same channel pairs for both multitaper and VAR estimates, suggesting genuine shared oscillatory activity rather than estimation artifacts. In several pairs, coherence rapidly decays at higher frequencies, with little structure above ~30–40 Hz, which is again consistent across both approaches.

Overall, the multitaper coherence tends to be slightly broader and smoother due to the 5-Hz bandwidth tapering, while the VAR-based coherence appears sharper and more localized in frequency. Despite these expected differences, the close qualitative agreement in peak frequencies and participating channel pairs supports the validity of the VAR model and indicates that both approaches are identifying the same underlying network interactions.

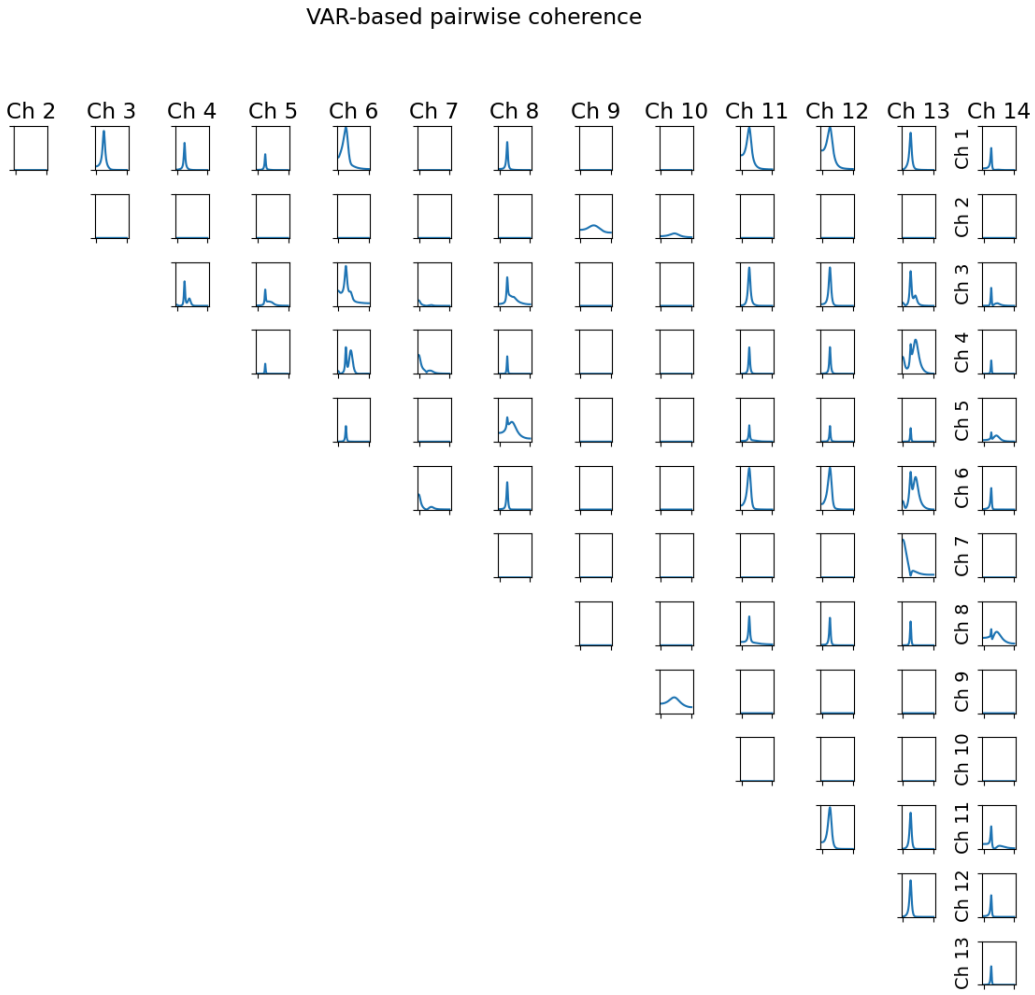
```
Xi = X[:, :, 0].T   # shape (N, nTrials)
```

```
Yj = X[:, :, 1].T
bandWidth = 5.0
NFFT = N

Pxx, Pyy, Pxy, XYphi, Cxy, F, nTapers = mtSpec(
    Xi, Yj,
    Fs=fs,
    bandWidth=bandWidth,
    NFFT=NFFT,
    RemoveTemporalMean=True,
    RemoveEnsembleMean=True,
    nTapers=[]
)

# Restrict to 0–100 Hz
mask = (F >= 0) & (F <= 100)
f_mt = F[mask]
nFreq_mt = np.sum(mask)

# Allocate coherence array: C_mt[f, i, j]
C_mt = np.zeros((nFreq_mt, nChns, nChns))

# Fill diagonal with NaNs (we're not plotting power on the diagonal)
C_mt[:, np.arange(nChns), np.arange(nChns)] = np.nan

# Compute multitaper coherence for each pair (i < j)
for i in range(nChns):
    for j in range(i+1, nChns):
        Xi = X[:, :, i].T  # (N, nTrials)
        Yj = X[:, :, j].T

        Pxx, Pyy, Pxy, XYphi, Cxy, F_tmp, nTapers_tmp = mtSpec(
            Xi, Yj,
            Fs=fs,
            bandWidth=bandWidth,
            NFFT=NFFT,
            RemoveTemporalMean=True,
            RemoveEnsembleMean=True,
            nTapers=[]
        )

        # Make sure frequency axis matches
        Cxy = Cxy[mask]      # Cxy is *squared* coherence in [0,1]
        # squared coherence
        C_mt[:, i, j] = Cxy

# Plot multitaper coherence in upper-triangular layout
fig, axes = plt.subplots(nChns, nChns, figsize=(12, 12), sharex=True, sharey=True)
```

```
for i in range(nChns):
    for j in range(nChns):
        ax = axes[i, j]

        if i < j:
            ax.plot(f_mt, C_mt[:, i, j])
            ax.set_ylim(0, 1)

            # improved labeling INSIDE the subplot
            ax.text(
                0.5, 0.85,
                f"Ch {i+1}-{j+1}",
                transform=ax.transAxes,
                ha="center", va="center", fontsize=7
            )

            # only label axes on outside edges
            if i == nChns - 1:
                ax.set_xlabel("Hz", fontsize=8)
            if j == 0:
                ax.set_ylabel("Coherence", fontsize=8)

        else:
            ax.axis('off')

for ax in axes[-1, :]:
    if ax.has_data():
        ax.set_xlabel('Frequency (Hz)')

plt.suptitle('Multitaper pairwise coherence (BW = 5 Hz)')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```


```
# S_f: (nFreq, nChns, nChns) from part (c)
nFreq, nChns, _ = S_f.shape

# coherence array: (nFreq, nChns, nChns)
C_var = np.zeros_like(S_f, dtype=float)

for i in range(nChns):
    for j in range(nChns):
        S_ij = S_f[:, i, j]
        S_ii = np.real(S_f[:, i, i])
        S_jj = np.real(S_f[:, j, j])
        denom = S_ii * S_jj
        C_var[:, i, j] = np.abs(S_ij)**2 / denom
freqs = np.linspace(0.0, 100.0, 301)
```

```python
# Plot upper-triangular matrix of VAR coherences
fig, axes = plt.subplots(nChns, nChns, figsize=(12, 12), sharex=True, sharey=True)
for i in range(nChns):
    for j in range(nChns):
        ax = axes[i, j]
        if i < j:
            ax.plot(freqs, C_var[:, i, j])
            ax.set_ylim(0, 1)
            if i == 0:
                ax.set_title(f'Ch {j+1}')
            if j == nChns-1:
                ax.set_ylabel(f'Ch {i+1}')
        else:
            ax.axis('off')

for ax in axes[-1, :]:
    if ax.has_data():
        ax.set_xlabel('Frequency (Hz)')

plt.suptitle('VAR-based pairwise coherence')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

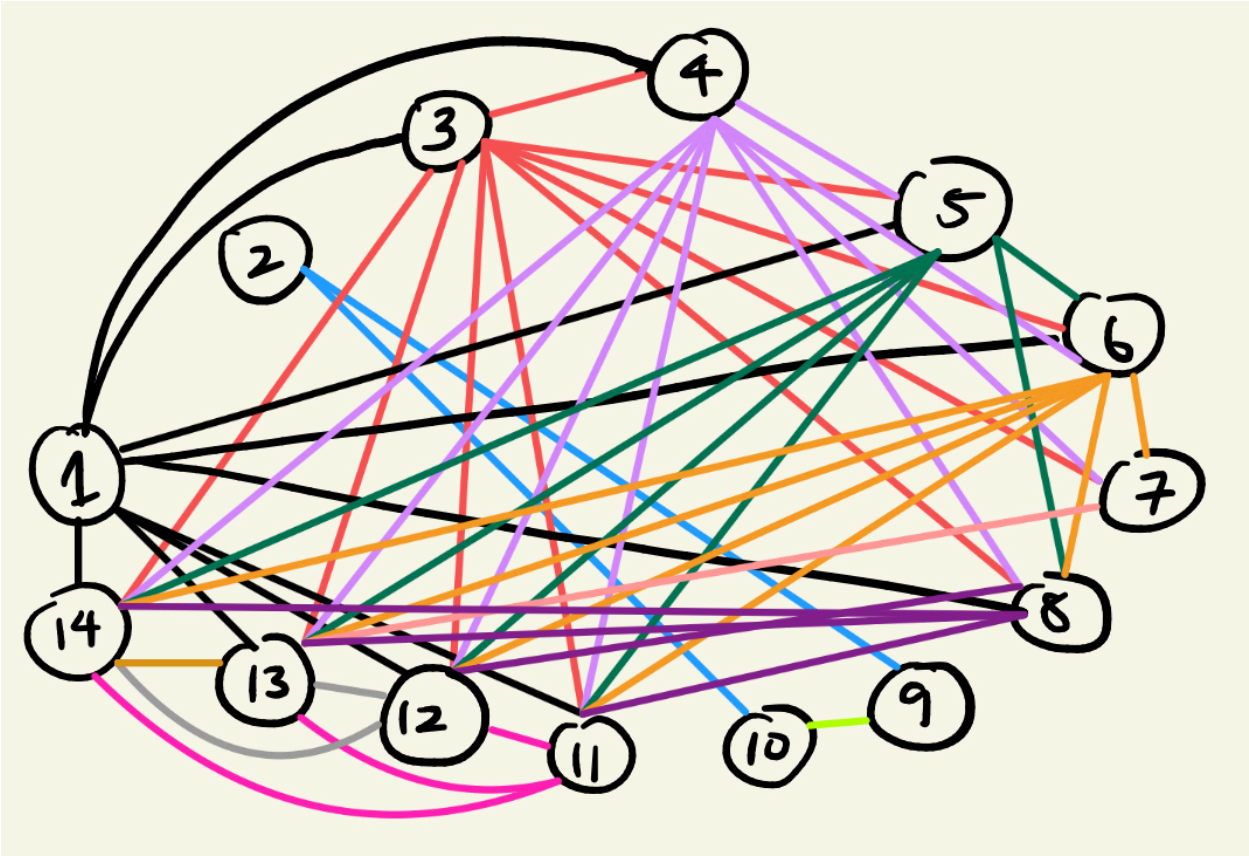


(e)

```
max_coh = np.max(C_var, axis=0)
threshold = 0.1
adj = max_coh >= threshold        # shape (nChns, nChns), True/False

# no self-connections on the diagonal
np.fill_diagonal(adj, False)
edges = []
for i in range(nChns):
    for j in range(i+1, nChns):      # i < j for undirected graph
        if adj[i, j]:
            edges.append((i+1, j+1))  # +1 to go from 0-based to channel numbers

print("Edges (channel pairs with max coherence >= 0.1):")
print(edges)
# 14 labeled nodes
# arranged in a circle, only connect if threshold is greater than >= 0.1
```

Edges (channel pairs with max coherence >= 0.1):

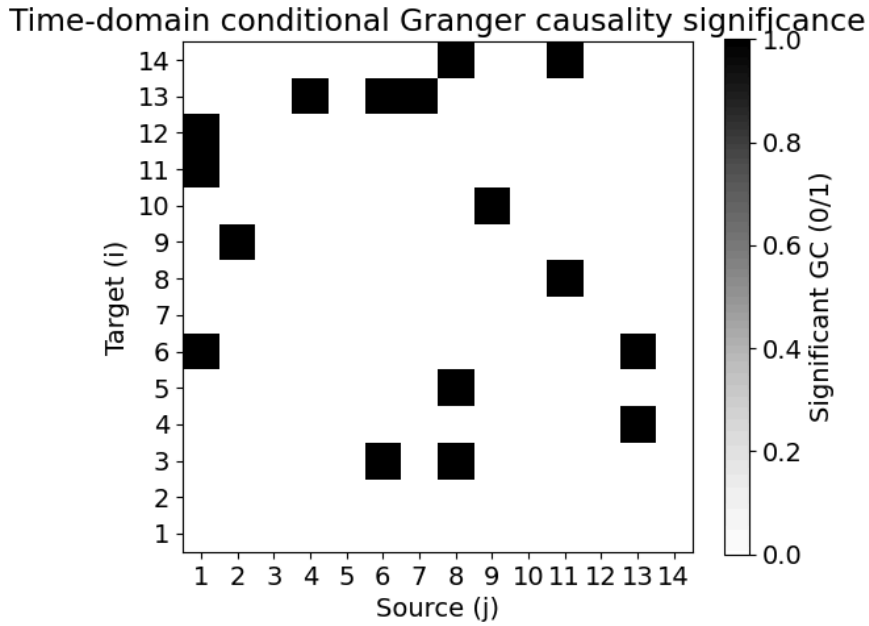
[(1, 3), (1, 4), (1, 5), (1, 6), (1, 8), (1, 11), (1, 12), (1, 13), (1, 14), (2, 9), (2, 10), (3, 4), (3, 5), (3, 6), (3, 7), (3, 8), (3, 11), (3, 12), (3, 13), (3, 14), (4, 5), (4, 6), (4, 7), (4, 8), (4, 11), (4, 12), (4, 13), (4, 14), (5, 6), (5, 8), (5, 11), (5, 12), (5, 13), (5, 14), (6, 7), (6, 8), (6, 11), (6, 12), (6, 13), (6, 14), (7, 13), (8, 11), (8, 12), (8, 13), (8, 14), (9, 10), (11, 12), (11, 13), (11, 14), (12, 13), (12, 14), (13, 14)]



(f) We can see that most significant links show the strongest GC at low frequencies, specifically 1 → 6, 1 → 12, and 7 → 13. With high frequencies, the GC is near zero for almost all links and there are no peaks at high frequencies. This may suggest that fast oscillations contribute little to directed information flow. We can also see that the Granger causality is basically negligible above ~ 30-40 Hz. Slow oscillatory dynamics may contribute to the majority of directed information flow in the network.

```
df = 0.001
alpha = 0.05
N = int(np.squeeze(N))
nTrials = int(np.squeeze(nTrials))
GC, GCf, CCF, pval, sig, F_gc = multivarGC(
    A_opt,
    SIGMA_opt,
    order=p_opt,
    Fs=fs,
    df=df,
    N=N,
    nTrials=nTrials,
    alpha=alpha
)
```

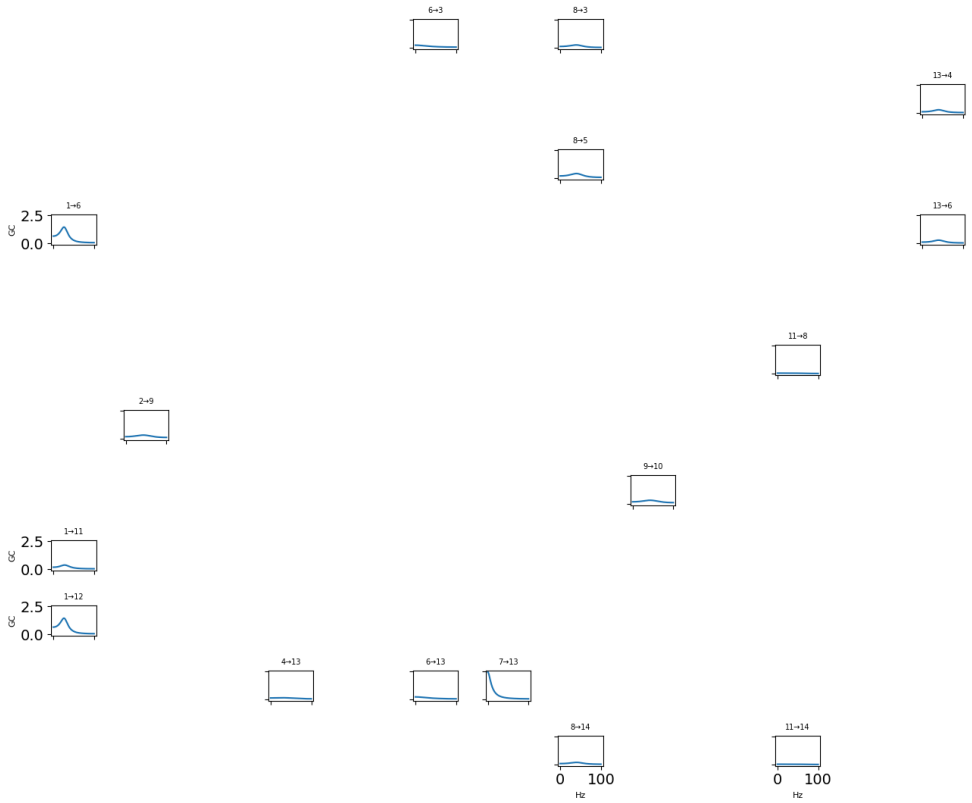
```
plt.figure(figsize=(6, 5))
plt.imshow(sig, origin='lower', cmap='Greys', vmin=0, vmax=1)
plt.colorbar(label='Significant GC (0/1)')
plt.xticks(np.arange(nChns), np.arange(1, nChns+1))
plt.yticks(np.arange(nChns), np.arange(1, nChns+1))
plt.xlabel('Source (j)')
plt.ylabel('Target (i)')
plt.title('Time-domain conditional Granger causality significance')
plt.tight_layout()
plt.show()
```




```
fig, axes = plt.subplots(nChns, nChns, figsize=(14, 14), sharex=True, sharey=True)
for i in range(nChns):       # target
    for j in range(nChns):   # source
        ax = axes[i, j]

        if i == j:
            ax.axis("off")
            continue

        if sig[i, j]:
            ax.plot(F_gc, GCf[:, i, j])
            ax.set_title(f"{j+1}→{i+1}", fontsize=7)
        else:
            ax.axis("off")

# label only outer edges
for ax in axes[-1, :]:
    if ax.has_data():
        ax.set_xlabel("Hz", fontsize=8)
```

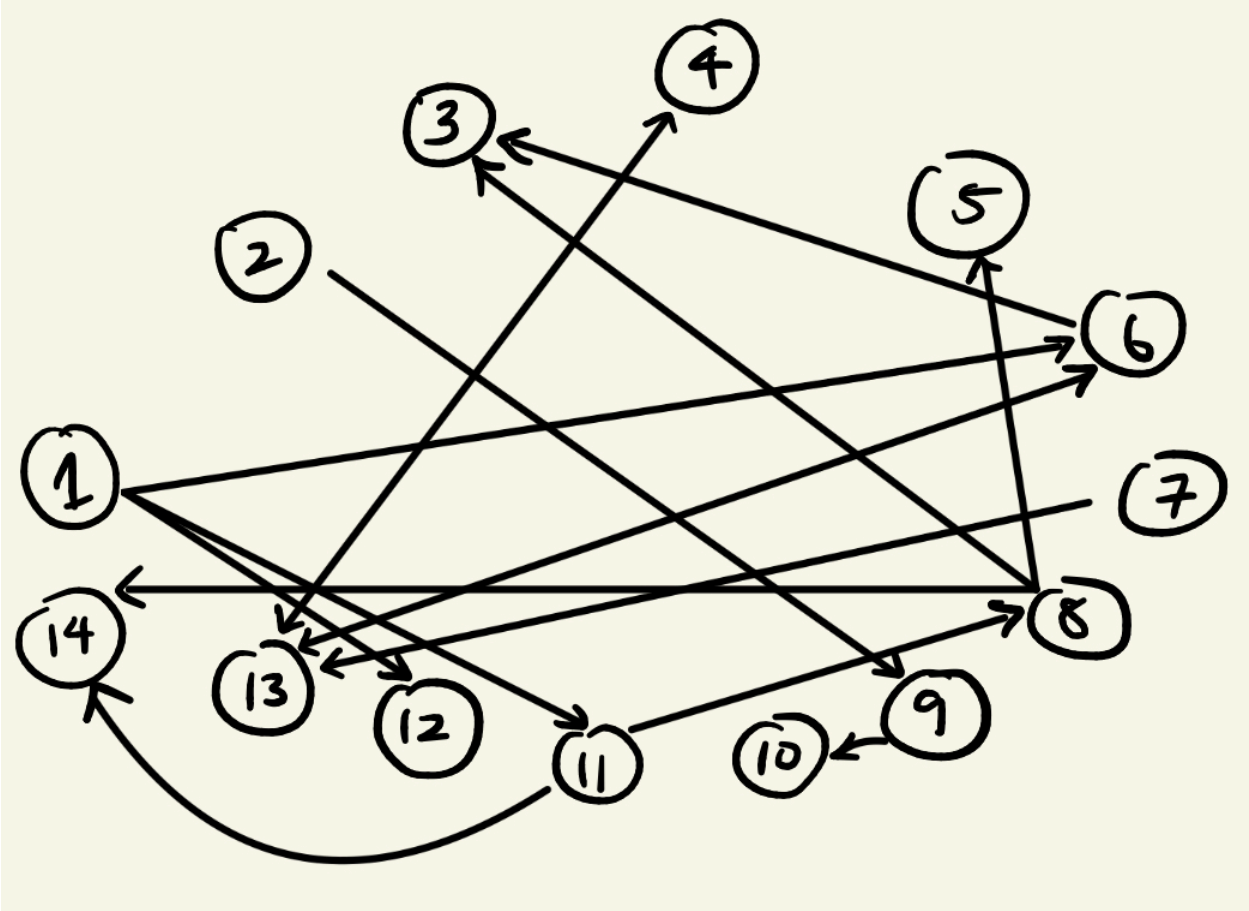
```
for ax in axes[:, 0]:
    if ax.has_data():
        ax.set_ylabel("GC", fontsize=8)

plt.suptitle("Spectral conditional Granger causality (only significant time-domain links)")
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Spectral conditional Granger causality (only significant time-domain links)




```
edges = [(j+1, i+1) for i in range(nChns) for j in range(nChns) if i != j and sig[i,j] ==
1]
edges
```

Edges = [(6, 3),(8, 3),(13, 4),(8, 5),(1, 6),(13, 6),(11, 8),(2, 9),(9, 10),(1, 11),(1, 12),(4, 13),(6, 13),(7, 13),(8, 14), (11, 14)]



(g) Both approaches, either from the pairwise spectral coherence and from time-domain conditional Granger causality measure show strong low-frequency coupling. In addition, both show similarities in connectivity between two channels. The main difference between the two is that spectral coherence is an undirected measure and only reflects oscillatory activity or undirected statistical coupling, while time-conditional Granger causality is directed and conditions on all other channels. This results in the Granger causality yielding a sparser network and becoming more selective.
Thus, the two approaches partially agree but they are not identical.

Exercise 3: functional and effective connectivity for neuronal spike train data

(a) Yes, the two neurons are functionally coupled. The null hypothesis of zero correlation can be rejected at the 0.05 significance level since $p < 0.05$. $r = 0.94$ supports this as the correlation coefficient is very close to 1, indicating shared activity (coupling).

```
# specifying the path to the .mat file
mat_file_path = 'two_neuron_spike_data.mat'

# load .mat file
data = sio.loadmat(r'c:\Users\Lyons\Desktop\NEUR2110\Final_Exam\two_neuron_spike_data.mat')

# getting a better idea of the shape of arrays
print(f"variables in .mat: {data.keys()}")
T = data['T'].squeeze()
s1 = data['stimes1'].squeeze()
```

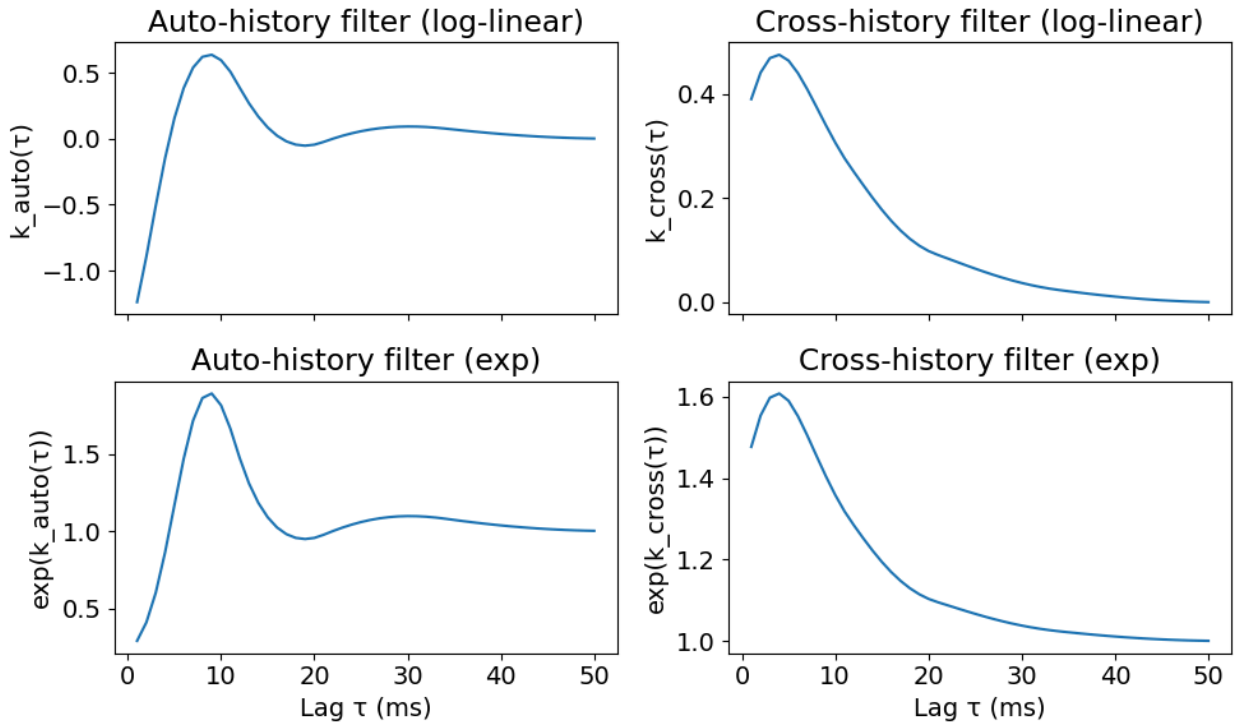
```
s2 = data['stimes2'].squeeze()
delta = .05 # s bin size
bins = np.arange(0, T + delta, delta) # bin edges from 0 to T

# histogram spike times into binned spike counts
counts1, _ = np.histogram(s1, bins=bins)
counts2, _ = np.histogram(s2, bins=bins)

# zero-lag Pearson cross-correlation coefficient between the two neurons.
r, p = pearsonr(counts1, counts2)
print(f"Pearson correlation coefficient: r = {r}, p-value = {p}")
```

Pearson correlation coefficient: r = 0.9385030578826028, p-value = 0.0

(b)

```
delta = 0.001  # 1 ms
edges = np.arange(0.0, T + delta, delta)
Nbins = len(edges) - 1
s1counts, _ = np.histogram(s1, bins=edges)
s2counts, _ = np.histogram(s2, bins=edges)

# Raised-cosine basis for history filters
nBasis_auto = 4
nBasis_cross = 4

ht = make_cosine_basis(nBasis_auto, 53, 10, offset=0,normalize=False)
ht = ht[:, :50]
maxHist = 50 # 50 ms history
ht = ht.T  # transpose to shape (time bins, basis functions)

def build_hist_design(spikes, basis_50xK):
    """Return (Nbins x K) design matrix from spikes using raised-cosine basis over past 50
bins."""
    N = len(spikes)
    K = basis_50xK.shape[1]
    Xh = np.zeros((N, K), dtype=float)
    for k in range(K):
        b = basis_50xK[:, k]               # length 50
        conv = np.convolve(spikes, b, mode='full')[:N]
        conv = np.roll(conv, 1)            # enforce causality (no current bin)
        conv[0] = 0.0
        Xh[:, k] = conv
    return Xh

# Build regressors: auto-history from neuron 1, cross-history from neuron 2
X_auto  = build_hist_design(s1counts, ht)    # (Nbins, 4)
X_cross = build_hist_design(s2counts, ht)    # (Nbins, 4)
X = np.hstack([X_auto, X_cross])              # (Nbins, 8)
# Add baseline column
X1 = np.hstack([np.ones((Nbins, 1), dtype=float), X])  # (Nbins, 9)
```

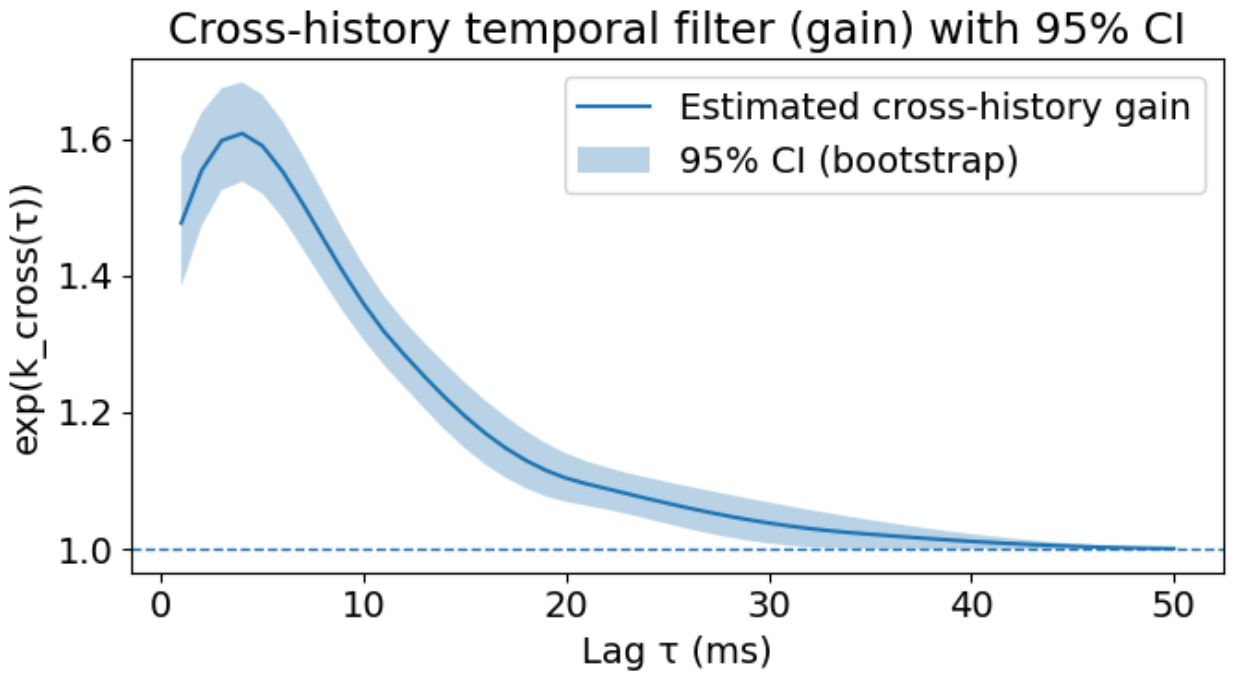
```
poisson_model = sm.GLM(s1counts, X1, family=Poisson(log()))
poisson_results = poisson_model.fit()
covb = poisson_results.normalized_cov_params
params = poisson_results.params

# Split coefficients: [baseline, auto(4), cross(4)]
B0 = params[0]
B_auto  = params[1:1+4]
B_cross = params[1+4:1+8]

# Reconstruct 50 ms filters (log-linear)
k_auto  = ht @ B_auto     # (50,)
k_cross = ht @ B_cross    # (50,)

# Exponentiated gain
gain_auto  = np.exp(k_auto)
gain_cross = np.exp(k_cross)

# Lag axis: 1..50 ms
lags_ms = np.arange(1, 51)

# ---- Plot ----
fig, ax = plt.subplots(2, 2, figsize=(10, 6), sharex=True)

ax[0,0].plot(lags_ms, k_auto)
ax[0,0].set_title("Auto-history filter (log-linear)")
ax[0,0].set_ylabel("k_auto(τ)")

ax[1,0].plot(lags_ms, gain_auto)
ax[1,0].set_title("Auto-history filter (exp)")
ax[1,0].set_xlabel("Lag τ (ms)")
ax[1,0].set_ylabel("exp(k_auto(τ))")

ax[0,1].plot(lags_ms, k_cross)
ax[0,1].set_title("Cross-history filter (log-linear)")
ax[0,1].set_ylabel("k_cross(τ)")

ax[1,1].plot(lags_ms, gain_cross)
ax[1,1].set_title("Cross-history filter (exp)")
ax[1,1].set_xlabel("Lag τ (ms)")
ax[1,1].set_ylabel("exp(k_cross(τ))")

plt.tight_layout()
plt.show()
```



(c)

```
BHat = params
nBasis = 4
cross_idx = np.arange(1 + nBasis, 1 + 2*nBasis)
newparams = np.random.multivariate_normal(BHat, covb,10000)
Bcross_boot = newparams[:, cross_idx]
k_cross_boot = Bcross_boot @ ht.T  # shape (10000, 50)
gain_cross_boot = np.exp(k_cross_boot)

Bcross_hat = BHat[cross_idx]             # (4,)
k_cross_hat = ht @ Bcross_hat            # (50,)
gain_cross_hat = np.exp(k_cross_hat)     # (50,)

# 95% confidence intervals from bootstrap samples
lower = np.percentile(gain_cross_boot, 2.5, axis=0)    # (50,)
upper = np.percentile(gain_cross_boot, 97.5, axis=0)   # (50,)

lags_ms = np.arange(1, 51)
plt.figure(figsize=(7, 4))
plt.plot(lags_ms, gain_cross_hat, label="Estimated cross-history gain")
plt.fill_between(lags_ms, lower, upper, alpha=0.3, label="95% CI
(bootstrap)")
plt.axhline(1.0, linewidth=1, linestyle="--")  # gain=1 means no effect
```

```
plt.xlabel("Lag τ (ms)")
plt.ylabel("exp(k_cross(τ))")
plt.title("Cross-history temporal filter (gain) with 95% CI")
plt.legend()
plt.tight_layout()
plt.show()
```

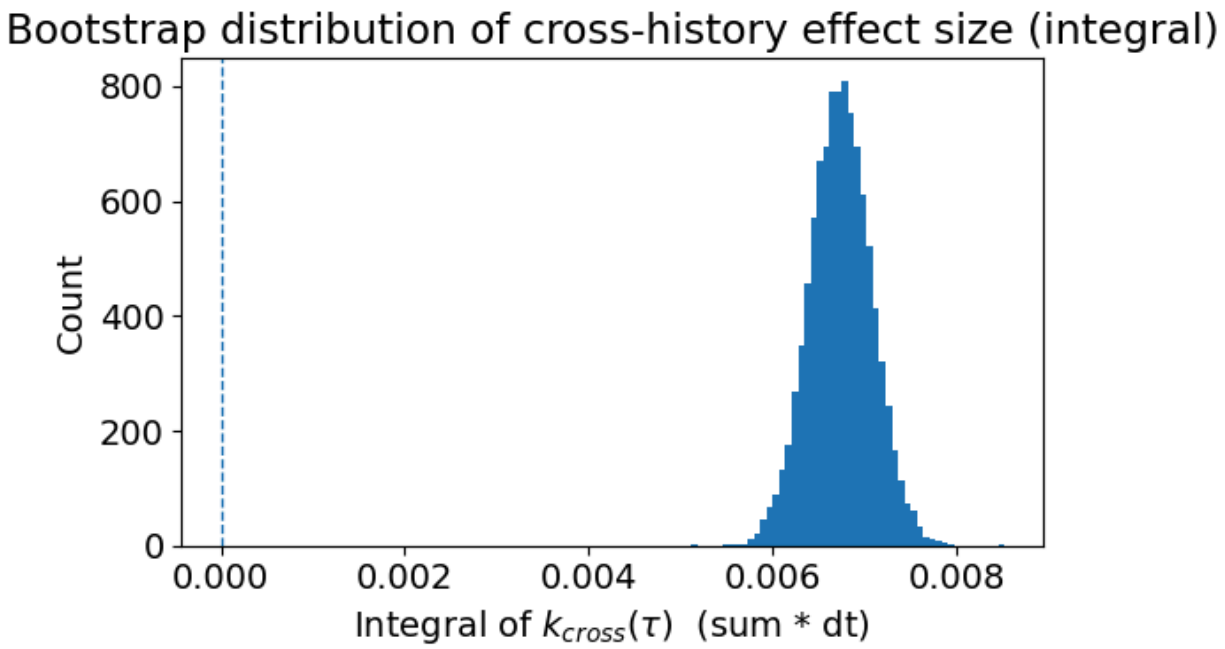


(d) The confidence interval does not include zero, since the CI is fully greater than zero. We can see that it provides evidence for a statistically significant time-causal effect from neuron 2 → 1 (since the 95% CI doesn't include zero). This indicates that past spiking activity of neuron 2 reliably improves the prediction of neuron 1's spiking at positive time lags, consistent with an excitatory cross-history effect under the GLM/point-process framework. However, despite this significant directed interaction, it is not possible to definitively conclude that neuron 2 provides monosynaptic input to neuron 1. The estimated coupling may instead reflect common input from unobserved neurons or populations, indirect or polysynaptic pathways, or other unmodeled network dynamics and shared covariates. Thus, the results support directed statistical coupling but do not constitute direct evidence of a monosynaptic connection.

```
integrals = np.sum(k_cross_boot, axis=1) * delta   # shape (10000,)

ci_low, ci_high = np.percentile(integrals, [2.5, 97.5])

print("95% CI for integral of cross-history filter (k_cross):")
print(f"[{ci_low:.6f}, {ci_high:.6f}]")
print("Includes 0?", (ci_low <= 0.0 <= ci_high))

#plot histogram of integrals
```

```
plt.figure(figsize=(6,4))
plt.hist(integrals, bins=50)
plt.axvline(0.0, linestyle="--", linewidth=1)
plt.xlabel(r'Integral of $k_{cross}(\tau)$  (sum * dt)')
plt.ylabel('Count')
plt.title('Bootstrap distribution of cross-history effect size
(integral)')
plt.tight_layout()
plt.show()
```

95% CI for integral of cross-history filter (k_cross):
[0.006072, 0.007397]
Includes 0? False



(e) Using the log-linear conditional intensity model, the conditional intensity of neuron 1 is: $\lambda_1(t) = \exp(\beta_0 + k_{\text{cross}}(\tau))$. Then, the baseline firing rate is therefore: $\lambda_0 = \exp(\beta_0)$, which gives λ_0 = 17.30 spikes/s. The probability of observing at least one spike in a 1-ms bin (delta = 0.001 s) is $p_0 = 1 - \exp(-\lambda_0 \Delta)$, yielding p_0 = 0.01715 per 1 ms bin. If neuron 2 were to fire an action potential, the cross history filter produces a gain on neuron 1's rate. The modulated rate at lag tau is $\lambda(\tau) = \lambda_0 g(\tau)$. We can see that this occurs when tau = 4 ms with g_max at around 1.608. At this lag, the firing rate is $\lambda_{\max} = \lambda_0 g_{\max}$, which is 27.81 spikes/s. This is an increase of delta(λ) = 10.52 spikes/s over baseline.

The spike probability in a 1-ms bin becomes $p_{\max} = 1 - \exp(-\lambda_{\max} \Delta)$, so p_max = 0.02743.
The change in spiking probability is just taking p_0 from p_max, which is 0.01028 per 1 ms bin.

```
dt = 0.001  # seconds (1 ms bin)
```

```
# Baseline (per-bin) intensity: expected spikes per 1 ms bin
beta0 = poisson_results.params[0]
lambda0_bin = np.exp(beta0)  # spikes/bin (bin = 1 ms)

# Convert to spikes/s
lambda0_hz = lambda0_bin / dt  # = lambda0_bin * 1000

# Spike probability in a 1 ms bin
p0 = 1 - np.exp(-lambda0_bin)

print("Baseline rate λ0 (spikes/s):", lambda0_hz)
print("Baseline P(spike in 1 ms):", p0)

# Cross-history gain (multiplicative)
gain_cross_hat = np.exp(k_cross_hat)
lags_ms = np.arange(1, 51)

tau_star_idx = np.argmax(gain_cross_hat)
tau_star_ms = lags_ms[tau_star_idx]
g_max = gain_cross_hat[tau_star_idx]

print("Max effect lag (ms):", tau_star_ms)
print("Max gain g_max:", g_max)

# Max-modulated intensity
lambda_max_bin = lambda0_bin * g_max
lambda_max_hz = lambda_max_bin / dt
delta_lambda_hz = lambda_max_hz - lambda0_hz

p_max = 1 - np.exp(-lambda_max_bin)
delta_p = p_max - p0

print("λ_max (spikes/s):", lambda_max_hz)
print("Δλ (spikes/s):", delta_lambda_hz)
print("p_max (1 ms):", p_max)
print("Δp (1 ms):", delta_p)
```

Baseline rate λ_0 (spikes/s): 17.296371837688717
Baseline P(spike in 1 ms): 0.017147648292047735
Max effect lag (ms): 4
Max gain g_max: 1.6080249885225812
λ_max (spikes/s): 27.8129981257817
Δλ (spikes/s): 10.516626288092983
p_max (1 ms): 0.02742977774856814
Δp (1 ms): 0.010282129456520406

Exercise 4: assessing spike-field amplitude- and phase-coupling with point process GLMs

(a) Using likelihood ratio (chi-squared) tests at a critical level of alpha = 0.05, we compared the nested models. Comparing the baseline firing rate model to the model including the instantaneous phase effect

(model 2) yielded a p-value of 0.0 (2 degrees of freedom). Since the p-value does not exceed 0.05, we reject the null hypothesis, indicating that the model including only the instantaneous phase effect provides a significantly better fit than the baseline firing rate model.

In contrast, comparing the model including only the instantaneous phase effect to the model including both instantaneous phase and amplitude yielded a p-value of 0.52 (1 degrees of freedom). Because this p-value exceeds 0.05, we fail to reject the null hypothesis and conclude that including amplitude in addition to phase does not significantly improve the model fit.

```
# specifying the path to the .mat file
mat_file_path = 'spike_LFP.mat'

# load .mat file
data = sio.loadmat(r'c:\Users\Lyons\Desktop\NEUR2110\Final_Exam\spike_LFP.mat')

# getting a better idea of the shape of arrays
print(f"variables in .mat: {data.keys()}")

dN = data['dN'].squeeze()  # spike counts
LFP = data['LFP']  # LFP signal
print(f"dN shape: {dN.shape}, LFP shape: {LFP.shape}")
fs = data['Fs'][0,0]        # sampling frequency

bp = [44, 46]
order = 100
fNQ = fs/2.0

# FIR bandpass coefficients
b = signal.firwin(numtaps=order+1, cutoff=bp, nyq=fNQ, pass_zero=False, window='hamming')

print("lfp length:", len(LFP))
print("fs:", fs)
# zero-phase filtering
lfp_bp = signal.filtfilt(b, 1, LFP, axis = 0)

# apply hilbert transform
z = signal.hilbert(lfp_bp)        # analytic signal
phi = np.angle(z)          # instantaneous phase (radians)
A = np.abs(z)              # instantaneous amplitude envelope

samples_per_bin = int(round(fs * delta))


# baseline only model
Y = dN.reshape(-1, 1, order='F') # Create a vector of responses
X0 =  np.ones((len(Y), 1)) # Create a matrix of predictors.
poisson_model = sm.GLM(Y, X0, family=Poisson()) # Fit the GLM.
poisson_results = poisson_model.fit()
b0 = poisson_results.params.reshape(-1, 1)
```

```
DEV0 = poisson_results.deviance


# baseline + phase model
phi_vec = phi.reshape(-1, order='F')    # (100000,)
X1 = np.column_stack([
    np.ones(len(Y)),
    np.cos(phi_vec),
    np.sin(phi_vec)
])

res1 = sm.GLM(Y, X1, family=sm.families.Poisson()).fit()
DEV1 = res1.deviance
# baseline + phase + amplitude model
A_vec   = A.reshape(-1, order='F')      # (100000,)
X2 = np.column_stack([
    np.ones(len(Y)),
    np.cos(phi_vec),
    np.sin(phi_vec),
    A_vec
])

res2 = sm.GLM(Y, X2, family=sm.families.Poisson()).fit()
DEV2 = res2.deviance
```

dN shape: (1000, 100), LFP shape: (1000, 100)
lfp length: 1000
fs: 1000

```
# compare deviances
alpha = 0.05
# is the phase term significant and does it improve the model over baseline?
D = DEV0 - DEV1
dof = res1.params.shape[0] - poisson_results.params.shape[0]
print(f"D = {D}")
print(f"%dof = {dof}")
pval = 1 - chi2.cdf(D,dof)
print(f"p-value = {pval}")

# is the amplitude term significant and does it improve the model over baseline + phase?
D = DEV1 - DEV2
dof = res2.params.shape[0] - res1.params.shape[0]
print(f"D = {D}")
print(f"%dof = {dof}")
pval = 1 - chi2.cdf(D,dof)
print(f"p-value = {pval}")
```

D = 200.62620606234123
%dof = 2
p-value = 0.0
D = 0.39897457232291345

%dof = 1
p-value = 0.5276193060564179

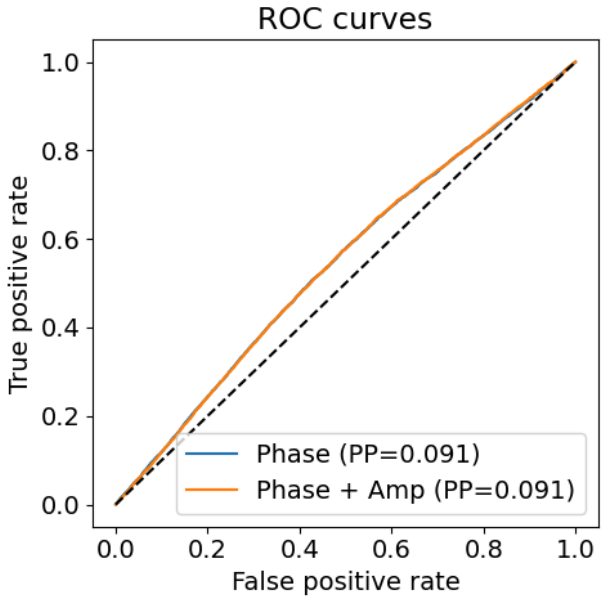
(b) (write something)

```
from sklearn.metrics import roc_curve, auc
# Binary response (0 or 1 per bin)
Ybin = Y.flatten()   # length 100000

# Predicted log-intensity
log_lambda_phase = X1 @ res1.params
log_lambda_amp   = X2 @ res2.params

# Predicted intensity
lambda_phase = np.exp(log_lambda_phase)
lambda_amp   = np.exp(log_lambda_amp)

# Phase model
fpr_phase, tpr_phase, _ = roc_curve(Ybin, lambda_phase)
AUC_phase = auc(fpr_phase, tpr_phase)
PP_phase = 2 * (AUC_phase - 0.5)

# Phase + amplitude model
fpr_amp, tpr_amp, _ = roc_curve(Ybin, lambda_amp)
AUC_amp = auc(fpr_amp, tpr_amp)
PP_amp = 2 * (AUC_amp - 0.5)

print("Phase-only model:")
print("  AUC =", AUC_phase)
print("  Predictive power =", PP_phase)

print("Phase + amplitude model:")
print("  AUC =", AUC_amp)
print("  Predictive power =", PP_amp)

print("Difference in predictive power:", PP_amp - PP_phase)
```

Phase-only model:
  AUC = 0.5056258945043641
  Predictive power = 0.011251789008728252
Phase + amplitude model:
  AUC = 0.5078103970820461
  Predictive power = 0.015620794164092233
Difference in predictive power: 0.004369005155363981

```
plt.figure(figsize=(5,5))
plt.plot(fpr_phase, tpr_phase, label=f'Phase (PP={PP_phase:.3f})')
plt.plot(fpr_amp, tpr_amp, label=f'Phase + Amp (PP={PP_amp:.3f})')
plt.plot([0,1],[0,1],'k--')
plt.xlabel('False positive rate')
```

```
plt.ylabel('True positive rate')
plt.legend()
plt.title('ROC curves')
plt.tight_layout()
plt.show()
```



Exercise 5: Gaussian linear state-space model estimation and Kalman filtering

```
# specifying the path to the .mat file
mat_file_path = 'GaussianLDS_CY_data.mat'

# load .mat file
data = sio.loadmat(r'c:\Users\Lyons\Desktop\NEUR2110\Final_Exam\GaussianLDS_XY_data.mat')

# getting a better idea of the shape of arrays
print(f"variables in .mat: {data.keys()}")
Xtest = data['Xtest']  # shape (T, n)
Ytest = data['Ytest']  # shape (T, m)
X = data['Xtrain']
Y = data['Ytrain']
print(f"Xtest shape: {Xtest.shape}, Ytest shape: {Ytest.shape}")
p, T = X.shape
q, Ttest = Y.shape

# Prepare regression matrices
X_prev = X[:, :-1]   # X_{t-1}
X_curr = X[:, 1:]    # X_t

# Estimate A
A_hat = X_curr @ X_prev.T @ np.linalg.inv(X_prev @ X_prev.T)

# Estimate Q
```

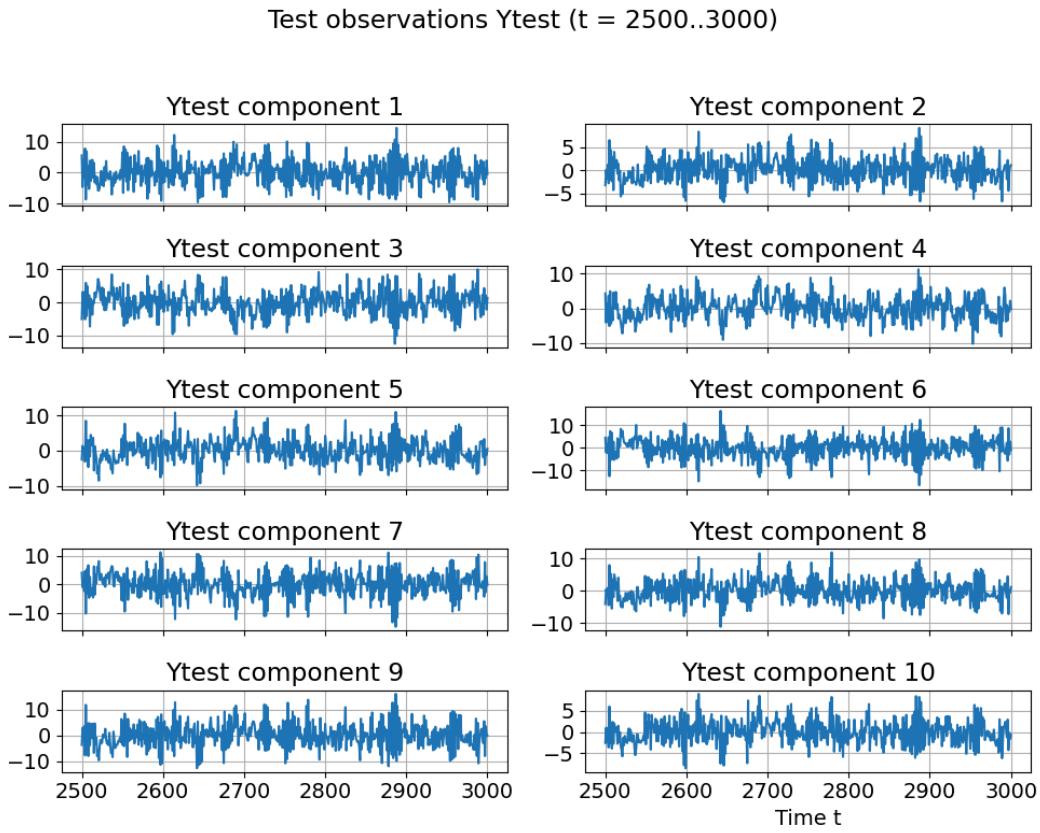
```
Eps = X_curr - A_hat @ X_prev
Q_hat = (E @ E.T) / (T - 1)

# Estimate B
B_hat = Y @ X.T @ np.linalg.inv(X @ X.T)
# Estimate R
Eta = Y - B_hat @ X
R_hat = (F @ F.T) / T

print("Estimated A:\n", A_hat)
print("Estimated Q:\n", Q_hat)
print("Estimated B:\n", B_hat)
print("Estimated R:\n", R_hat)
```

```
variables in .mat: dict_keys(['__header__', '__version__', '__globals__', 'Xtest', 'Xtrain', 'Ytest', 'Ytrain'])
Xtest shape: (2, 4000), Ytest shape: (10, 4000)
Estimated A:
 [[ 0.70776203 -0.00832629]
 [ 0.0102773  -0.8968908 ]]
Estimated Q:
 [[ 0.99908672 -0.00281839]
 [-0.00281839  0.97435207]]
Estimated B:
 [[ 1.64759667 -1.70938398]
 [ 1.06768015  1.03090988]
 [-1.8534193   1.27332175]
 [ 1.93697777 -1.05805198]
 [ 1.68449142  1.09410067]
 [-1.75849502 -1.82108659]
 [-1.74398759 -1.69296193]
 [ 1.37958226  1.32224825]
 [ 1.64337945  1.9416111 ]
 [ 1.1799802   1.04064649]]
Estimated R:
 [[ 9.45382326e-01  3.84118328e-03  8.97030477e-03 -1.82793710e-02
   6.72768166e-03  3.75630390e-03 -5.46272651e-03 -5.43908164e-03
  -1.51112717e-02 -3.35233562e-03]
 [ 3.84118328e-03  9.99308309e-01 -1.23965369e-02  2.32544121e-03
   1.96618558e-02 -2.29545540e-02 -9.08805139e-03 -3.00803431e-02
...
   1.02164183e+00  4.74963830e-04]
 [-3.35233562e-03  1.63547124e-02 -1.03133954e-02  7.60794606e-03
  -1.92598236e-03  1.73044039e-02  2.20980297e-02  1.26725948e-02
   4.74963830e-04  1.03921922e+00]]
```

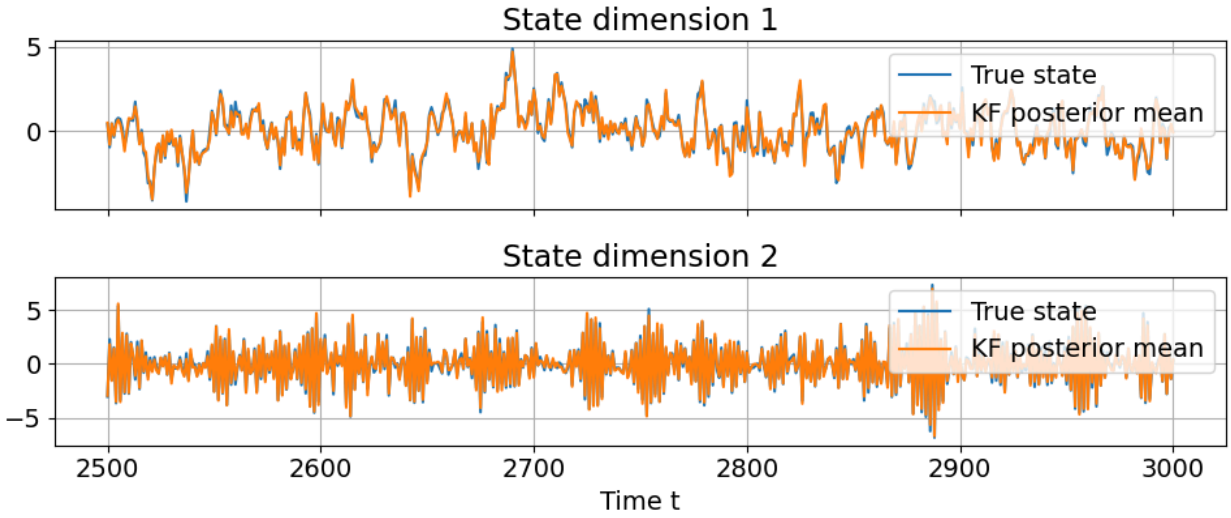
(b)

```
# Kalman filter on test data
x_filt = np.zeros((p, Ttest))          # posterior means: E[X_t | y_1:t]
P_filt = np.zeros((p, p, Ttest))       # posterior covariances

I = np.eye(p)

# Initialization (simple, standard choices)
x_prev_filt = np.zeros((p,))           # E[X_0]
P_prev_filt = np.eye(p) * 1.0          # Cov[X_0] (can also use Q_hat)

for t in range(Ttest):
```

```
    # 1) Predict
    x_pred = A_hat @ x_prev_filt
    P_pred = A_hat @ P_prev_filt @ A_hat.T + Q_hat

    # 2) Update with observation y_t
    y_t = Ytest[:, t]
    S = B_hat @ P_pred @ B_hat.T + R_hat              # innovation covariance (q×q)
    K = P_pred @ B_hat.T @ np.linalg.inv(S)           # Kalman gain (p×q)

    innov = y_t - (B_hat @ x_pred)
    x_new = x_pred + K @ innov
    P_new = (I - K @ B_hat) @ P_pred

    # store
    x_filt[:, t] = x_new
    P_filt[:, :, t] = P_new

    # step
    x_prev_filt = x_new
    P_prev_filt = P_new

# Plot Ytest components for t = 2500 - 3000
t0, t1 = 2500, 3000
tt = np.arange(t0, t1+1)

fig, axes = plt.subplots(5, 2, figsize=(10, 8), sharex=True)
axes = axes.ravel()

for i in range(q):
    axes[i].plot(tt, Ytest[i, t0:t1+1])
    axes[i].set_title(f"Ytest component {i+1}")
    axes[i].grid(True)

axes[-1].set_xlabel("Time t")
plt.suptitle("Test observations Ytest (t = 2500..3000)")
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```




```
# plot the true states of Xtest components for t = 2500 - 3000 & inferred states from
Kalman filter
fig, axes = plt.subplots(2, 1, figsize=(10, 5), sharex=True)

for k in range(p):
    axes[k].plot(tt, Xtest[k, t0:t1+1], label="True state")
    axes[k].plot(tt, x_filt[k, t0:t1+1], label="KF posterior mean")
    axes[k].set_title(f"State dimension {k+1}")
    axes[k].grid(True)
    axes[k].legend()

axes[-1].set_xlabel("Time t")
plt.suptitle("True states vs Kalman-filtered posterior means (t = 2500..3000)")
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```


True states vs Kalman-filtered posterior means (t = 2500..3000)
State dimension 1
5
0
True state
KF posterior mean
State dimension 2
5
0
−5
True state
KF posterior mean
2500
2600
2700
2800
2900
3000
Time t